

High Availability with Kubespray

Kubernetes Production · Lesson 7

Built from the official documentation

`kubernetes-sigs/kubespray` · ha-mode · kube-vip · etcd · reliability

Learning Objectives

By the end of this lesson you will be able to:

- Explain what an **HA control plane** means in Kubespray
- Choose between **localhost LB**, **external LB** and **kube-vip** for the API
- Configure **kube-vip** in **ARP** or **BGP** mode for a control-plane VIP
- Pick an **etcd deployment type** and **stacked vs external** topology
- Understand and **tune node-failure detection** and failover speed

HA = make the two single-points-of-failure tiers — **etcd** and the **kube-apiserver** — resilient.

7.1 What "HA Control Plane" Means

Two Tiers Made Highly Available

Kubespray makes two critical components HA:

- **etcd cluster** — already **HA-ready with multiple peers**: every client receives the **complete peer list**, so no extra proxy is needed.
- **kube-apiserver** — needs a **reverse-proxy / load-balancer layer** in front of the apiserver instances to become HA.

Production rule of thumb: **≥ 3** control-plane / etcd nodes, **odd** count for etcd quorum (3 → survives 1 failure, 5 → survives 2).

7.2 API Server Load Balancing

Three Ways to Reach the API

Option	What it is
Localhost LB (default)	Per-node nginx reverse proxy on non-master nodes
External LB	One external VIP / domain all nodes point to
kube-vip	A virtual IP managed inside the cluster (ARP or BGP)

- etcd is HA by itself; this layer is **only about the apiserver**.
- Without localhost LB, use the **kube-vip role** or your **own load balancer**.

Workers and **kubelet** / **kube-proxy** must always have a stable apiserver endpoint.

Internal / localhost Load Balancer (Default)

- By default, non-master nodes run a **local nginx reverse proxy** that proxies API-server connections.
- Enabled by `loadbalancer_apiserver_localhost` — defaults to `True`.
- Only **kubelet** and **kube-proxy** on non-master nodes use the local proxy.
- Port `loadbalancer_apiserver_port` — defaults to `kube_apiserver_port`.

Less efficient than a dedicated LB (more health checks), but **practical** when no external LB infrastructure exists — hence the default.

External Load Balancer

Point all nodes at one external VIP + domain name:

```
apiserver_loadbalancer_domain_name: "my-apiserver-lb.example.com"
loadbalancer_apiserver:
  address: <VIP>
  port: 8383
```

- The domain name is added to `/etc/hosts` of every `k8s_cluster` host.
- It is also **wired into the generated self-signed TLS certificates**, so the cert is valid for the LB name.

7.3 kube-vip — Control-Plane VIP

What kube-vip Provides

- A **virtual IP and load balancer** for **both** the control plane (HA cluster) **and** Kubernetes Services of type LoadBalancer.
- **No external hardware or software** required.
- **Control-plane HA** via `kube_vip_controlplane_enabled` + `kube_vip_address`.
- **Service LoadBalancer** (optional) via `kube_vip_services_enabled`.
- Advertises the VIP in **ARP** (layer-2) or **BGP** (routed) mode.

Set `kube_proxy_strict_arp: true` when `kube_proxy_mode` is `ipvs` **and** kube-vip ARP is enabled.

kube-vip — ARP Mode (Layer-2)

```
kube_proxy_strict_arp: true
kube_vip_enabled: true
kube_vip_controlplane_enabled: true
kube_vip_address: 10.42.42.42
loadbalancer_apiserver:
  address: "{{ kube_vip_address }}"
  port: 6443
# kube_vip_interface: ens160
kube_vip_arp_enabled: true
```

- Simplest mode — VIP advertised via **ARP** on the L2 segment.
- `kube_vip_interface` pins the VIP to a specific NIC.

kube-vip — BGP Mode (Routed)

```
kube_vip_bgp_enabled: true
kube_vip_local_as: 65000
kube_vip_bgp_routerid: 192.168.0.2
kube_vip_bgppeers:
- 192.168.0.10:65000::false
- 192.168.0.11:65000::false
```

- Use BGP for **routed / multi-subnet** fabrics instead of L2 ARP.
- Peers as `address:as::password-flag`; pair with `kube_vip_address` for the VIP.

7.4 etcd — Deployment & Topology

etcd Deployment Types

`etcd_deployment_type` selects how etcd runs:

Value	What it does
<code>host</code> (default)	etcd as a systemd service on the host
<code>kubeadm</code>	etcd as a static Pod on control-plane hosts (experimental, new deploys)
<code>docker</code>	etcd in docker containers (container manager = docker)

Members live in the inventory **etcd group**; metrics via `etcd_metrics_port` (e.g. `2381`).

Stacked vs External etcd

- **Stacked** — etcd **co-located on the control-plane nodes** (common Kubespray default; `host` or `kubeadm`).
- **External** — **dedicated etcd nodes**, separate from the control plane.
- For externally load-balanced etcd, override:
`etcd_access_addresses`, `etcd_client_url`, `etcd_cert_alt_names`,
`etcd_cert_alt_ips`.

Always use an **odd number ≥ 3** of etcd members to keep **quorum** through a failure.

7.5 Failure Detection & Reliability

How a Node Failure Is Detected

Parameter	Default	Role
kubelet <code>--node-status-update-frequency</code>	10s	kubelet reports status
ctrl-mgr <code>--node-monitor-period</code>	5s	how often statuses are checked
<code>--node-monitor-grace-period</code>	40s	healthy if updated in this window
<code>nodeStatusUpdateRetry</code>	5	hardcoded retries
<code>--default-not-ready-toleration-seconds</code>	300s	wait before eviction
<code>--default-unreachable-toleration-seconds</code>	300s	wait before eviction

Past the grace period the node is unhealthy → pods evicted; **kube-proxy** drops its endpoints from Service iptables.

Tuning Failover Speed

- **Fast** (~20-50s eviction): `update-frequency=4s`, `monitor-period=2s`, `grace-period=20s`, toleration `30s`.
- **Medium** (~3 min): `update-frequency=20s`, `grace-period=2m`, toleration `60s`.
- **Slow** (~6 min, fewer false positives): `update-frequency=1m`, `grace-period=5m`, toleration `60s`.

Real-world detection is **slower than the theory** (network latency, apiserver delay, etcd commit time) — favour stability in production.

Key Takeaways

- HA = HA **etcd** (≥ 3 peers, odd) + HA **kube-apiserver** (a LB layer in front).
- API LB options: **localhost LB** (default, per-node nginx), **external LB** (`loadbalancer_apiserver` + domain name), or **kube-vip**.
- **kube-vip** gives a control-plane VIP (`kube_vip_controlplane_enabled`) in **ARP** (`kube_vip_arp_enabled`) or **BGP** (`kube_vip_bgp_enabled`) mode.
- etcd runs as `host` / `kubeadm` / `docker`, **stacked** or **external**.
- Failover speed is set by kubelet / controller-manager timers — fast/med/slow.

End of Lesson 7

Next: Lesson 8 — Deploying Applications

`loadbalancer_apiserver` · `kube_vip_*` · `etcd_deployment_type`